

LAB1: 조합논리회로 실습

실험 후 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/09/14

대상: AND, OR, XOR 게이트/전가산기/4비트 가산기/4비트 감산기/4비트 비교기
/MUX/DEMUX/인코더/디코더/세그먼트 디코더

검증 환경: VS Code VaporView, Vivado 2026.1, FPGA Spartan

보고서 및 실험의 범위

기본 논리 게이트부터 산술 연산기, 멀티플렉서, 디코더 등 총 10종의 조합논리회로(실험 01~10)를 대상으로 Vivado 시뮬레이션 및 FPGA 보드 위 검증을 수행하였다.

소스 식별

프로젝트: lab1

기준 커밋: 818e934b77da883164b22f5207b5192c259f3e65

실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab1.git

제출 태그: v1.0

실제 보드에서는 두 입력을 1번, 2번 버튼으로 설정하고 AND는 1번 LED, OR는 2번 LED, XOR는 3번 LED에 연결하여 관찰하였

다. 입력 순서는 00->01->10->11로 하였고 각각 X->2,3->2,3->1,2번 LED가 점등되는 것을 확인하였다. 이를 통해 사전 검증 및 Vivado 시뮬레이션 결과와 일치하는 것을 확인하였다.

3. 실험2: 전가산기

설계와 예상 동작 요약

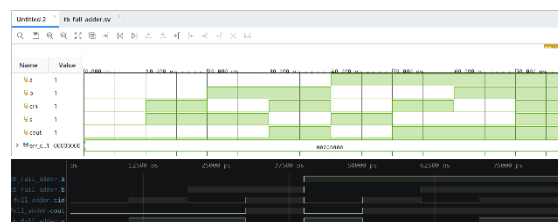
본 회로는 두 반가산기와 OR연산으로 구성되어 있다. 첫 반가산기는 a와 b를 더하고, 두번째 반가산기는 그 합에 cin을 더하는 두 개의 반가산기의 구조로 되어있다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS full_adder cases=8
$finish called at time : 80 ns
```

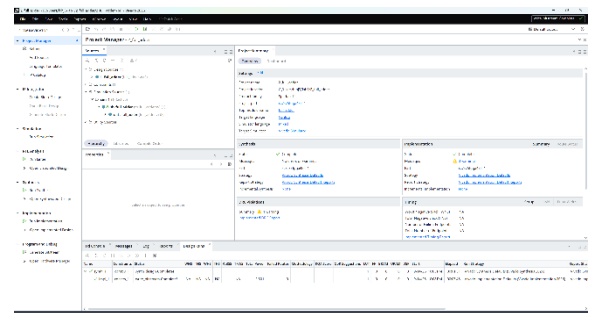
LAB1_PASS full_adder cases=8
C:\Users\hij3\OneDrive\444\lab1\lab1_02_full_adder\sim\tb_full_adder.sv:79: \$finish called at 80ns00 (ns)
시뮬레이션 실행 완료. 원리표, 자기검사, 파형으로 회로의 작동 여부를 확인하세요.
Wave C:\Users\hij3\OneDrive\바탕 화면\lab1\lab1_02_full_adder\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.

vivado에서 simulation 한 결과 case=8, 80ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 비록 cout과 s의 위치가 바뀌었지만, 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 입력 a를 1번 버튼, b를 2번 버튼, cin을 3번 버튼으로 설정하고 s는 2번 LED, cout은 1번 LED로 설정하였다. 000->001->010->100->011->110->101->111의 순서로 실험을 수행하였고 각각 00->01 3번->10 3번->11의 결과가 나타났고 이를 통해 검증 결과와 같음을 확인하였다.

4. 실험3: 4비트 가산기

설계와 예상 동작 요약

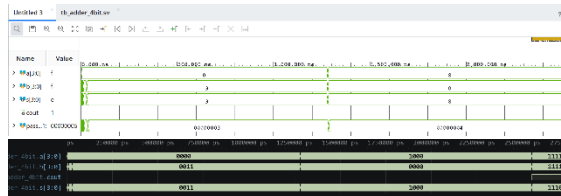
본 회로는 두 4비트 부호 없는 입력 a,b를 더하여 하위 4비트 합 s와 캐리 cout을 출력한다. {cout,s}=a+b의 관계를 갖는다. 합이 16 이상이라면 cout=1이다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS adder_4bit cases=5
$finish called at time : 2570 ns
```

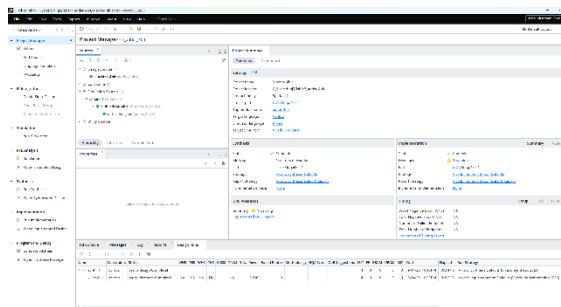
LAB1_PASS adder_4bit cases=5
C:\Users\hij3\OneDrive\444\lab1\lab1_03_adder4\sim\tb_adder_4bit.sv:68: \$finish called at 2570ns00 (ns)
시뮬레이션 실행 완료. 원리표, 자기검사, 파형으로 회로의 작동 여부를 확인하세요.
Wave C:\Users\hij3\OneDrive\바탕 화면\lab1\lab1_03_adder4\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.

vivado에서 simulation 한 결과 case=5, 2570ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 dip switch 1~4번을 a[3:0], dip switch 5~8번을 b[3:0], 1번 LED를 cout, 2~5번 LED를 s로 설정하였다. A와 b의 합이 16 미만이라면 1번 LED가 점등되지 않는 것과 16 이상일 때 점등되는 것을 확인하였으며 모든 스위치를 1로 올

렸을 때는 마지막 LED를 제외한 모든 LED가 점등되는 것을 확인하였다. 이는 사전 검증 결과 및 시뮬레이션 결과에서 확인한 바와 일치한다.

5. 실험4: 4비트 감산기

설계와 예상 동작 요약

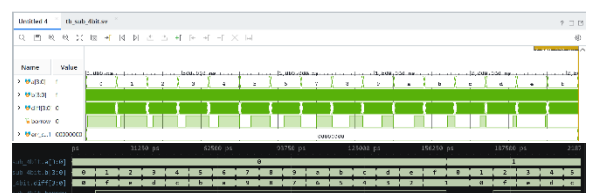
본 회로는 4비트 가산기와 다르게 두 입력을 각각 5비트로 확장한 뒤 a-b를 계산하여 하위 4비트 차이 diff와 빌림 출력 borrow를 생성한다. a가 b보다 크거나 같으면 borrow는 0, 반대의 경우에는 1이다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS sub_4bit cases=256
$finish called at time : 2560 ns
```

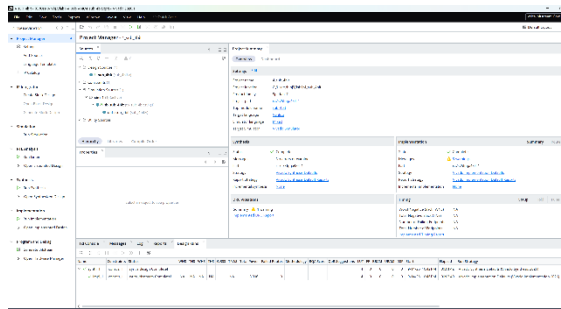
시뮬레이션 실행 완료. 정리로, 자기점서-리형으로 학원의 정답 여부를 확인하세요.
MWE C:\Users\hjh3\OneDrive\바탕 화면\lab1\lab1_04 subtractor\4bit subtractor\4bit subtractor.vcd
Terminal will be reused by tasks, press any key to close it.

vivado에서 simulation 한 결과 case=256, 2560ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. Vivado의 경우 조금 더 전체적인 파형, vscode에서는 a=0과 1에 대한 구간에 대해서만 보였다. 해당 구간 내에서 두 파형이 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 dip switch 1~4번을 a[3:0], dip switch 5~8번을 b[3:0], 1번 LED를 빌림수, LED 2~5를 diff로 설정하였다. a보다 b가 큰 경우에는 1번 LED가 점등되었으며, 두 수가 같은 경우에는 어떠한 LED도 점등되지 않았고, a가 b보다 큰 경우에는 1번 LED가 점등되지 않는 것을 확인하였다. 이러한 부분은 사전 검증 및 시뮬레이션 결과와 보드 구현 결과가 일치함을 보여준다.

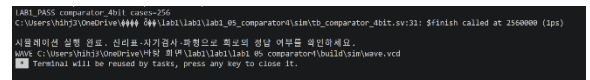
6. 실험5: 4비트 크기 비교기

설계와 예상 동작 요약

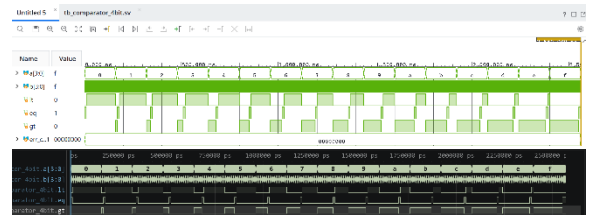
본 회로는 두 4비트 부호 없는 입력의 크기를 비교하여 비교 상태 lt, eq, gt를 출력한다. a < b이면 lt만 1, 같으면 eq만 1, 나머지 경우에는 gt만 1이 된다.

Vivado 시뮬레이션 및 사전 결과 비교

LAB1_PASS comparator_4bit cases=256
\$finish called at time : 2560 ns :

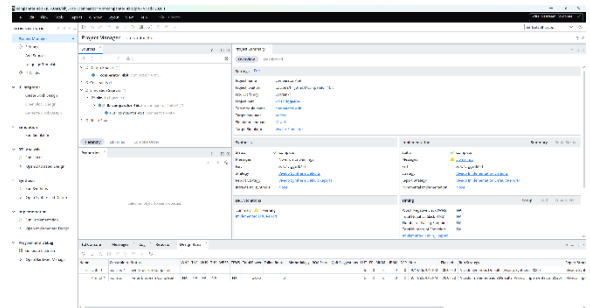


vivado에서 simulation 한 결과 case=256, 2560ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 dip switch 1~4번을 a,

5~8번을 b로 설정하고 LED 1번을 gt, 2번을 eq, 3번을 lt로 설정하였다. a>b인 상황에서는 1번 LED, a=b인 경우에는 2번 LED, a<b인 경우에는 3번 LED가 점등되는 것을 확인하였다. 이는 gt, lt의 순서를 제외하고 사전 검증 결과 및 시뮬레이션 결과에서 확인한 바와 일치한다.

7. 실험6: 4:1 멀티플렉서

설계와 예상 동작 요약

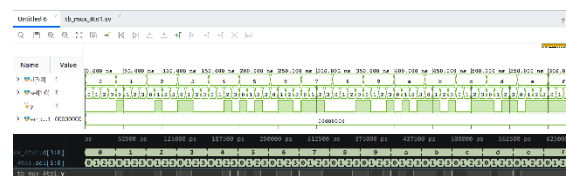
본 회로는 네 데이터 비트 d[3:0] 중 선택 신호 sel[1:0]이 지정한 비트를 출력 y로 전달한다. 선택값 00, 01, 10, 11은 각각 d[0], d[1], d[2], d[3]에 대응하며, $y=d[sel]$ 의 관계식이 성립한다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS mux_4to1 cases=64
$finish called at time : 640 ns
```

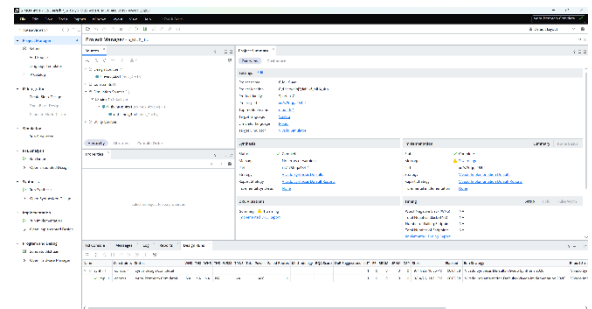
```
LAB1_PASS mux_4to1 cases=64
C:\Users\lab1\Documents\lab1\lab1_mux_4to1\lab1_mux_4to1.vhdl: Finish called at 640000 (ns)
시뮬레이션 실행 완료. 합리화, 컴파일 및 생성으로 새로운 회로가 생성되었습니다.
C:\Users\lab1\Documents\lab1\lab1_mux_4to1\lab1_mux_4to1.vhdl: Finish called at 640000 (ns)
Terminal will be reused by tasks, press any key to close it.
```

vivado에서 simulation 한 결과 case=64, 640ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

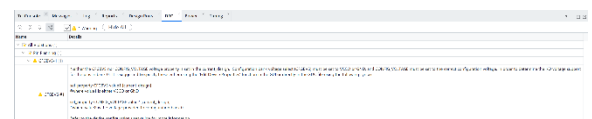


위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 DIP 스위치 2개를 선택 입력으로, 버튼 4개를 데이터 입력으로 사용하였다. DIP 스위치의 조합에 따라 네 버튼 중 하나의 입력이 출력으로 전달되도록 구성하였다. Dip switch를 00으로 조작하였을 때는 1번 버튼, 01일 때는 2번 버튼, 10일 때는 3번 버튼, 11일 때는 4번 버튼에서만 LED가 점등되는 것을 확인하였다. (다만, 사전 검증 과정에서 설계한 xdc 파일과는 다르다. 실제 보드 구현에서는 조원이 교안대로 작성한 RTL 코드 및 xdc 파일을 사용하였기 때문이다.)

8. 실험7: 1:8 디멀티플렉서

설계와 예상 동작 요약

본 회로는 단일 입력 din을 선택 신호 sel[2:0]이 지정한 출력으로 전달하며, 나머지 출력은 0으로 유지한다. Din=0이면 선

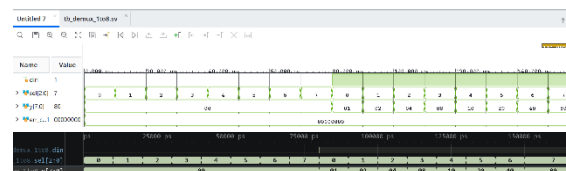
택값과 관계없이 y[7:0]=00000000 이고, din=1이면 선택된 비트 하나만 1이 된다. 따라서 din=1에서 din=1에서 sel을 0부터 7까지 증가시키면 출력은 16진수로 01, 02, 04, 08, 10, 20, 40, 80 순서가 되어야 한다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS demux_lto8 cases=16
$finish called at time : 160 ns
```

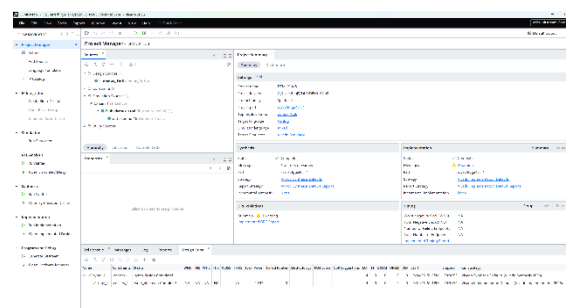
LAB1_PASS demux_lto8 cases=16
C:\Users\hhb3\OneDrive\4444\4444\lab1\lab1_07_demux_lto8.vi:32: \$finish called at 160000 (ps)
시뮬레이션 실행 완료. 진리표-자기검사-회형으로 회로의 정당 여부를 확인하십시오.
파일 C:\Users\hhb3\OneDrive\4444\4444\lab1\lab1_07_demux_lto8.vi:wave.vcd
Terminal will be reused by tasks, press any key to close it.

vivado에서 simulation 한 결과 case=16, 160ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

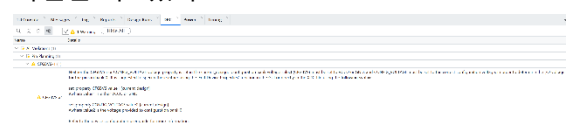


위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고

1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 din을 1번 버튼으로, sel을 dip switch로 설정하였으며 출력은 LED로 확인하였다. Din 버튼을 누르지 않으면 dip switch를 아무리 조작하여도 LED는 점등되지 않고 din 버튼을 누르면 dip switch를 통해 구현한 이진수에 따라 LED가 점등되는 것을 확인하였다. 이는 사전 검증 및 시뮬레이션 결과에서 확인한 바와 일치한다.

9. 실험8: 8:3 인코더

설계와 예상 동작 요약

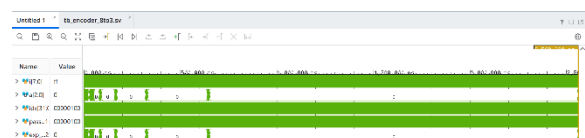
본 회로는 8비트 입력 중 하나의 비트만 활성화된 경우 이를 3비트 코드로 변환한다. 입력 i[7]부터 i[0]까지 각각 출력 000부터 111에 대응한다. 전체 입력이 0이거나 여러 비트가 동시에 활성화되면 기본값 000을 출력하므로 우선순위 인코더와는 구분된다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB1_PASS encoder_8to3 cases=256
$finish called at time : 2560 ns
```

LAB1_PASS encoder_8to3 cases=256
C:\Users\hhb3\OneDrive\4444\4444\lab1\lab1_08_encoder_8to3.vi:54: \$finish called at 2560000 (ps)
시뮬레이션 실행 완료. 진리표-자기검사-회형으로 회로의 정당 여부를 확인하십시오.
파일 C:\Users\hhb3\OneDrive\4444\4444\lab1\lab1_08_encoder_8to3.vi:wave.vcd
Terminal will be reused by tasks, press any key to close it.

vivado에서 simulation 한 결과 case=256, 2560ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

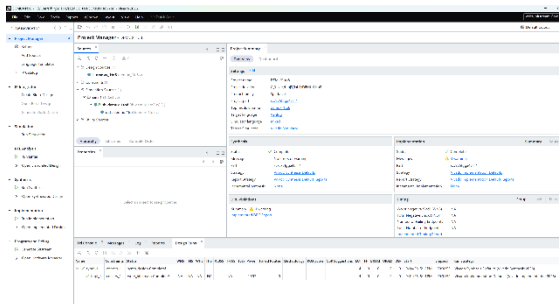




위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다.

Vivado의 경우 조금 더 전체적인 파형, vscode에서는 초반 부분의 가시성을 위해 확대한 사진이다. 초반에는 a가 0이 아닌 경우가 많지만 점점 0인 경우가 많아지는 것을 통해 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 1~8번 버튼을 1, 1~3번 LED를 통해 3비트 출력으로 설정하였다. 1번 버튼을 눌렀을 때는 모든 LED가 소등되어있었고, 버튼 번호를 증가시킬 때마다 LED가 나타내는 이진수가 증가함을 확인하였다. 또한, 버튼 2개 이상을 동시에 눌렀을 때 LED가 점등되지 않는 것을 확인하였다. 이는 사전 검증 결과 및 시뮬레이션 결과에서 확인한 바와 일치한다.

10. 실험9: 3:8 디코더

설계와 예상 동작 요약

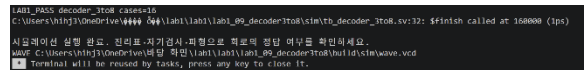
n-to-m 디코더의 목적은 n 입력 변수들로부터 2^n 개의 최소항들을(혹은 이보다 더 적은) 생성하는 것이다. [1]

본 회로는 3비트 입력 in[2:0]을 해석하여 8개의 출력 중 해당 번호의 비트 하나만 활성화하는 조합논리회로이다. 선택되지 않은 출력은 모두 0이 된다. 따라서 입력 000, 011, 111에 대한 출력은 각각 00000001, 00001000, 10000000이다. 아래는 각 입력에 대한 출력의 표다.

000	00000001
001	00000010
010	00000100
011	00001000
100	00010000
101	00100000
110	01000000
111	10000000

Vivado 시뮬레이션 및 사전 결과 비교

\$finish called at time : 80 ns



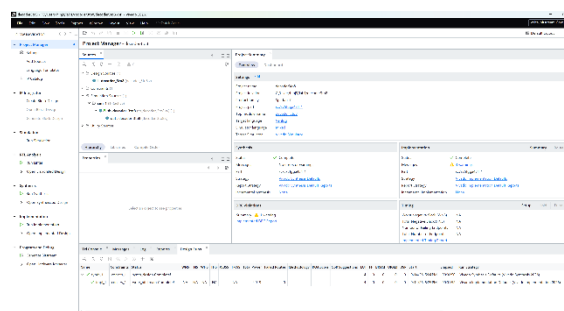
우선 vs code를 통한 사전 검증에서는 en 신호를 통해 en=1일 때만 출력이 될 수 있도록 설정하였다. 다만 실험 진행 과정에서 en 신호가 필요 없을 것으로 판단하여 en 신호에 대한 코드를 삭제하였다. vivado에서 simulation 한 결과 case=8, 80ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과인 case=16, 160ns에서 종료된 것과 다를

확인할 수 있다. 이는 en 신호가 삭제됨에 따라 case 및 시간이 절반으로 줄어든 것이다.

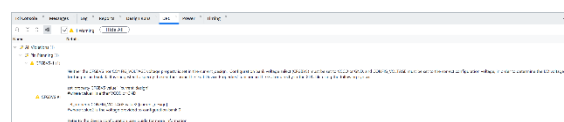


위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. vivado에는 en 신호가 없기 때문에 vscode에서 en이 1이 된 80ns부터 160ns까지의 파형을 비교하면 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

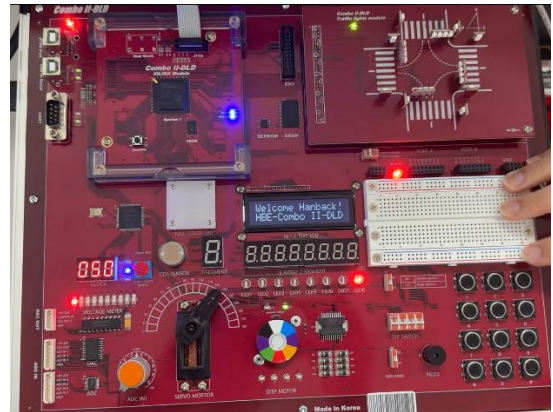
실제 보드 구현 및 동작 분석

실제 보드에서는 입력을 버튼으로, 출력을 LED로 설정하였다. 아무 버튼도 누르지 않았을 때는 8번 LED, 버튼을 차례대로 누를 때마다 7, 6, 5번 순서대로 앞의 LED만 점등되고 나머지 LED는 꺼지는 것을 확인하였다. 이는 사전 검증에서 en=1로 했을 때

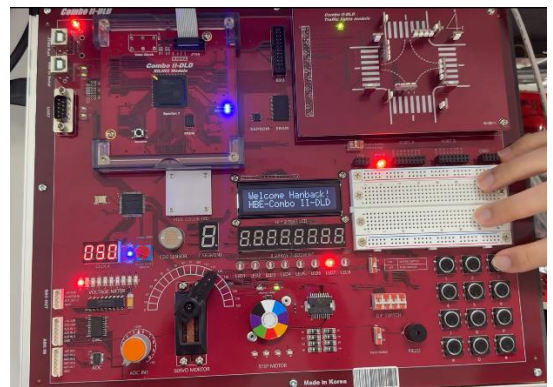
와 시뮬레이션 결과와 일치하는 것을 확인할 수 있다.

아래는 실제 보드 위에 구현한 사진 및 조작에 따른 출력의 표다.

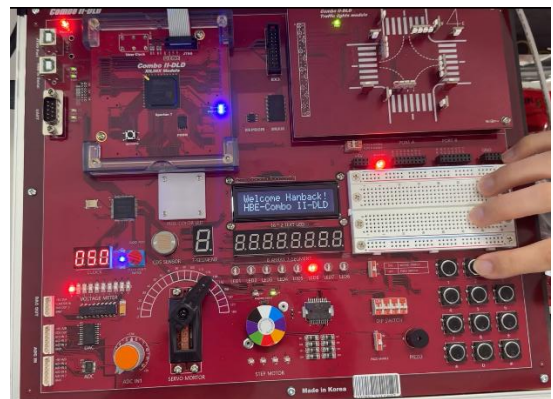
입력: X/출력: 00000001(8번 LED)



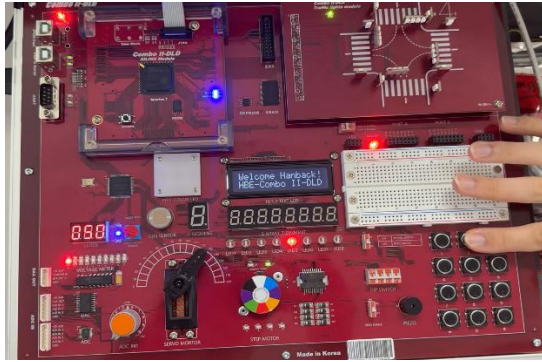
입력: 3번 버튼/출력: 00000010(7번 LED)



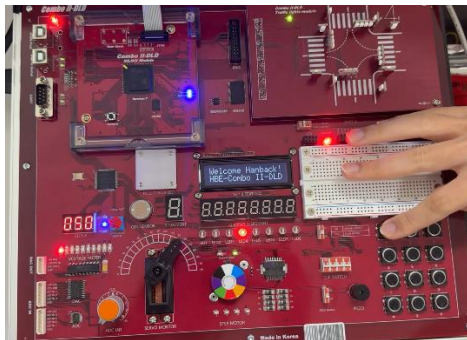
입력: 2번 버튼/출력: 00000100(6번 LED)



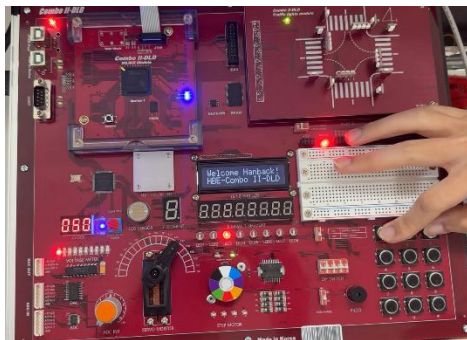
입력: 2,3번 버튼/출력: 00001000(5번 LED)



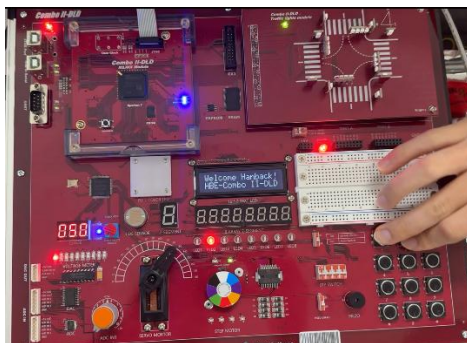
입력: 1번 버튼/출력: 00010000(4번 LED)



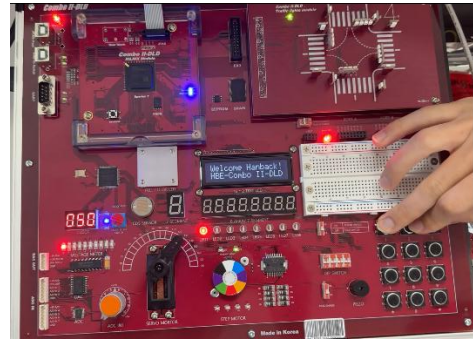
입력: 1,3번 버튼/출력: 00100000(3번 LED)



입력: 1,2번 버튼/출력: 01000000(2번 LED)



입력: 1,2,3번 버튼/출력: 10000000(1번 LED)



누른 버튼	점등되는 LED
X	8번
3	7번
2	6번
2,3	5번
1	4번
1,3	3번
1,2	2번
1,2,3	1번

11. 실험10: 7세그먼트 디코더

설계와 예상 동작 요약

본 회로는 4비트 입력을 숫자 0~9 및 문자 A~F 세그먼트 패턴으로 변환한다. 출력 seg_data[7:0]의 순서는 {a,b,c,d,e,f,g,dp}이며 1이 해당 막대의 점등을 의미한다. 입력 0, 1, 8, F에 대한 출력은 각각 16진수 FC, 60, FE, 8E가 되어야 한다.

Vivado 시뮬레이션 및 사전 결과 비교

LAB1_PASS seven_seg cases=16

\$finish called at time : 160 ns

LAB1_PASS seven_seg cases=16
C:\Users\shj\OneDrive\Documents\lab1_10_seven_seg\src\lab1_10_seven_seg.vv:81: finish called at 160000 (1ps)
시뮬레이션 실패. 원인을 찾기 위해, 파형으로 오류의 원인을 확인하십시오.
wave C:\Users\shj\OneDrive\Documents\lab1_10_seven_seg\src\lab1_10_seven_seg.vv:81: finish called at 160000 (1ps)
Terminal will be reused by tasks, press any key to close it.

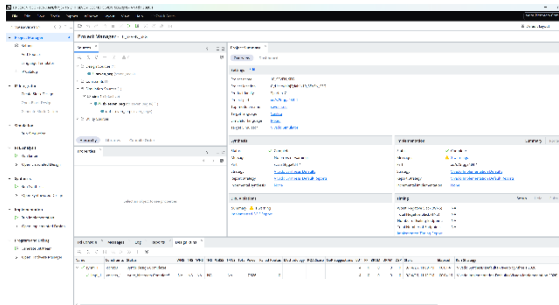
vivado에서 simulation 한 결과 case=16, 160ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



	05	25000 ps	50000 ps	75000 ps	100000 ps	125000 ps	150000 ps
sig_bcd[3:0]	8	1	2	3	4	5	6
sig_data[7:0]	fc	60	da	f2	60	b0	b0

위 사진은 vivado simulation의 파형, 아래는 vscode에서 시뮬레이션한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.

구현 회로는 4비트 입력을 0~F의 세그먼트 패턴으로 변환한다. 입력 포트 이름은 bcd지만, 10~15도 A,b,C,d,E,F로 정의되어 있으므로, 십진수 0~9만을 처리하는 회로로 해석하지 않았다. 출력 비트 순서는 a,b,c,d,e,f,g,dp이다.



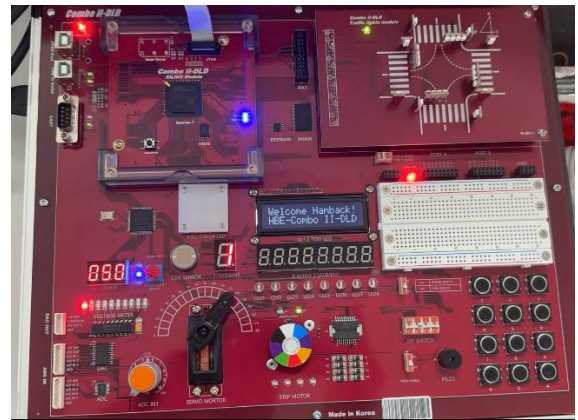
DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

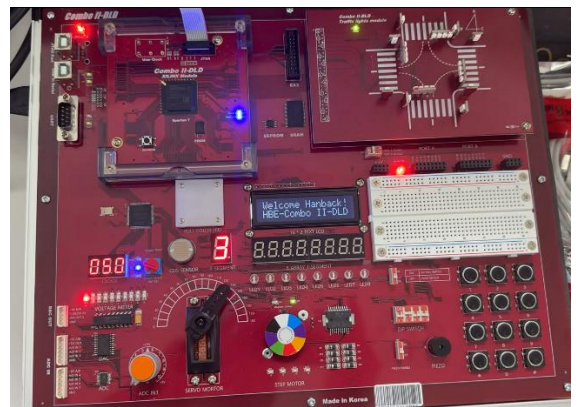
실제 보드에서는 dip switch 1~4번을 bcd에 대응하여 설정하였다. 4번을 켜올 때는 세그먼트에 1이 표시되고, 3,4번을 켜올 때는 3, 모두 켜올 때는 F가 표시되는 것을 확인할 수 있다. 이는 사전 검증 및 시뮬레이션 결과와 같은 것을 확인할 수 있다. 아래는 실제 보드 위에 구현한 사진 및 조

작에 따른 출력의 표다.

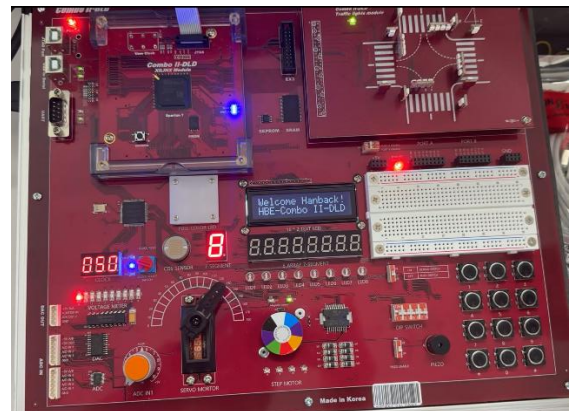
입력: 4번 dip switch/출력: 1(세그먼트)



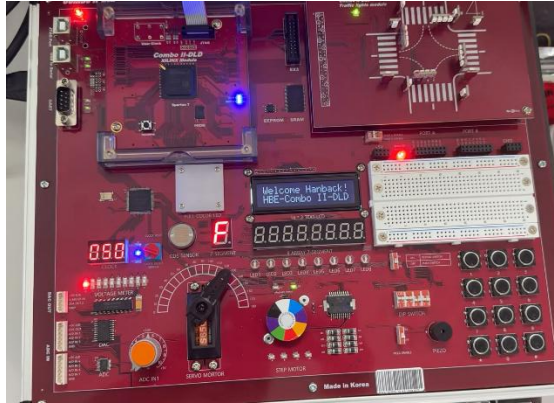
입력: 3, 4번 dip switch/출력: 3(세그먼트)



입력: 1번 dip switch/출력: 8(세그먼트)



입력: 1,2,3,4번 dip switch/출력: F(세그먼트)



아래는 조작에 따른 출력의 표다.

Dip switch	Segment 표시
X	0
4	1
3	2
3,4	3
2	4
2,4	5
2,3	6
2,3,4	7
1	8
1,4	9
1,3	A
1,3,4	b
1,2	C
1,2,4	d
1,2,3	E
1,2,3,4	F

12. 종합 고찰 및 결론

종합 고찰

본 실험에서는 기본 논리 게이트, 산술 연산 회로, 비교기, MUX와 DEMUX, 인코더와 디코더 및 7세그먼트 디코더를 설계하고 시뮬레이션과 fpga 보드 구현을 통해 조합논리회로의 동작을 검토하였다. 각 회로의 입력 조건과 출력값을 비교하는 과정

에서, 조합논리회로는 현재 입력의 조합에 따라 출력이 결정된다는 공통적인 특성을 확인할 수 있었다. 시뮬레이션에서는 입력과 출력의 논리값을 한 눈에 볼 수 있었지만, 실제 보드 구현에서는 경우의 수에 따른 출력만을 볼 수 있기에 제한된 경우에서만 진행하였다. 또한, 입출력과 연결된 버튼 및 스위치 버튼과 LED에 대해 정확히 파악하는 것이 중요하였다.

결론

본 실험을 통해 여러 조합논리회로를 RTL로 설계하고, 시뮬레이션, 합성, 구현, 비트스트림 생성 및 FPGA 보드 위 구현으로 이어지는 설계 과정을 수행하였다. 특히 실험 9와 실험 10의 시연을 통해 논리적인 입출력 관계를 실제 보드의 LED 및 세그먼트 표시와 연결하여 검토하였다.

13.참고 문헌

[1] M. Morris Mano, Charles R. Kime, Tom Martin, 『디지털논리와 컴퓨터설계』, 제5판, 양세양·최명렬·최석림·하경재 역, 퍼스트북, 2016, p. 120.